

MODULE-2

JAVASCRIPT / ECMASCRIPT 6

PROF. SUNITA D RATHOD

INTRODUCTION TO ES6

- ES6, or ECMAScript 6, is a major update to the JavaScript language specification standardized by ECMA International.
- It was officially released in June 2015 and brought several significant enhancements and new features to JavaScript, making it more powerful and expressive.
- European Computer Manufacturers Association (ECMAScript) or (ES) is a standard for scripting languages like JavaScript, ActionScript and JScript.

FEATURES OF ES6

- **let and const Declarations:** let allows you to declare block-scoped variables, replacing var in many cases. const allows you to declare variables whose value cannot be re-assigned once set.

```
console.log(number)
// undefined

var number = 10

console.log(number)
// 10
```

```
console.log(number)

let number = 10
// or const number = 10

console.log(number)
```

- you can access a variable declared with var before declaration without errors, but you cannot do the same with let or const.

FEATURES OF ES6

- **Arrow Functions:** Arrow functions provide a concise syntax for writing anonymous functions. They lexically bind **this** and are generally more concise than traditional function expressions.

Before Arrow:

```
hello = function() {  
  return "Hello World!";  
}
```

With Arrow Function:

```
hello = () => {  
  return "Hello World!";  
}
```

FEATURES OF ES6

- **Template Literals:** Template literals provide a way to embed expressions inside string literals, using backticks (``) instead of single or double quotes. This makes string interpolation and multi-line strings much easier.

SYNTAX : ``string text \${expression} string text``

Eg:

```
const a = 5;
const b = 10;
console.log(`Fifteen is ${a + b} and
not ${2 * a + b}.`);
// "Fifteen is 15 and
// not 20."
```

FEATURES OF ES6

- **Enhanced Object Literals:** Object literals now support shorthand for defining properties and methods, computed property names, and method definitions using function.

```
// variable declaration
var name = "Duke";
var color = "Brown";
var age = 5;

// Using Object Literal Enhancement
// Combines all variables into a dog object
var dog = {name, color, age};
console.log(dog);
```

FEATURES OF ES6

- **Destructuring Assignment:** Destructuring allows you to easily extract array elements or object properties and assign them to variables.

```
let a, b, rest;
[a, b] = [10, 20];

console.log(a);
// Expected output: 10

console.log(b);
// Expected output: 20

[a, b, ...rest] = [10, 20, 30, 40, 50];

console.log(rest);
// Expected output: Array [30, 40, 50]
```

FEATURES OF ES6

- **Modules:** ES6 introduces a standardized module format to JavaScript, allowing you to define modules and import/export functionality between them.

```
export { capitalize, roundToDecimalPlace };
```

filepath/main.js

SEPARATE FILE

```
import { capitalize, roundToDecimalPlace } from './main';

function displayTotal(name, total) {
  return `${capitalize(name)}, your total cost is: ${roundToDecimalPlace(total)}`;
}
```


FEATURES OF ES6

- **Classes:** ES6 introduces a class syntax that provides more familiar object-oriented programming (OOP) syntax and inheritance hierarchies.
- **Promises:** Promises provide a cleaner and more flexible way to handle asynchronous operations compared to callbacks.

FEATURES OF ES6

- **Iterators and Generators:** Iterators provide a standard way to iterate over data structures (like arrays), while generators provide a way to create iterators.
- **New Built-in Methods:** ES6 added many new methods to built-in objects like `'Array.prototype.includes'`, `'String.prototype.startsWith'`, `'Object.assign'`, etc.

DIFFERENCE BETWEEN ES5 AND ES6

FEATURE	ES-5	ES-6
Variable Declaration	Uses ' var ' keyword.	Introduces ' let ' and ' const ' for block-scoped variables.
Function Definition	Uses ' function ' keyword.	Introduces arrow functions ('=>') for concise function syntax.
Modules	No built-in module support.	Introduces native support for modules (' import ' and ' export ' keywords).

DIFFERENCE BETWEEN ES5 AND ES6

FEATURE	ES-5	ES-6
Classes	Uses prototype-based inheritance.	Introduces 'class' syntax for OOP, including constructor and inheritance.
Arrow Functions	Not available.	Provides a more concise syntax for defining functions.
Template Literals	Not available.	Introduces backtick (`) syntax for string interpolation and multiline strings.

DIFFERENCE BETWEEN ES5 AND ES6

FEATURE	ES-5	ES-6
Destructuring	Not available.	Allows for extracting values from arrays or objects into distinct variables.
Promises	Not standardized.	Introduces ' Promise ' object for asynchronous operations.
Built-in Methods	Fewer built-in methods (' forEach ', etc.).	Adds new methods (' Array.includes ', ' String.startsWith ', etc.) to built-in objects.

DIFFERENCE BETWEEN ES5 AND ES6

FEATURE	ES-5	ES-6
Strict Mode	Optional (' use strict ' ; declaration).	Applies strict mode by default.
Compatibility	Widely supported across browsers.	Newer syntax and features may require transpilation for broader support.
Adoption	Widely used prior to ES6 release.	Adoption as the modern standard for JavaScript development.

VARIABLES

'let' and **'const'** Declarations:

'let': Allows you to declare variables that are block-scoped (scoped to the nearest enclosing block '{ ... }'). This means variables declared with **'let'** are not accessible outside of their containing block.

'LET' EXAMPLE

```
let x = 10;
```

```
if (true) {
```

```
  let y = 20;
```

```
  console.log(x); // 10
```

```
  console.log(y); // 20
```

```
}
```

```
console.log(x); // 10
```

```
console.log(y); // ReferenceError: y is not defined
```


CONST

const: Allows you to declare constants, which are block-scoped like variables declared with `let`, but their value cannot be reassigned once set.

EXAMPLE :

```
const PI = 3.14;
```

```
PI = 3.14159; // TypeError: Assignment to constant variable
```

Key Differences and Usage:

Block Scope: Both `let` and `const` are block-scoped, unlike `var` which is function-scoped.

Hoisting: Variables declared with `let` and `const` are hoisted to the top of their block, but not initialized. Accessing them before initialization results in a `ReferenceError`.

Temporal Dead Zone (TDZ): Variables declared with `let` and `const` have a TDZ from the start of the block until the declaration is processed, where accessing them would also result in a `ReferenceError`.

ES6 LOOPS

1. for...of Loop

The for...of loop provides an easy way to iterate over iterable objects (arrays, strings, maps, sets, etc.):

```
// Iterating over an array
const numbers = [1, 2, 3, 4, 5];
for (const num of numbers) {
    console.log(num);
}

// Iterating over a string
const message = "Hello";
for (const char of message) {
    console.log(char);
}
```

ES6 LOOPS

2. for...in Loop

The for...in loop iterates over the enumerable properties of an object:

```
const person = { name: 'Alice', age: 30 };

for (const key in person) {
  console.log(`${key}: ${person[key]}`);
}
```

ES6 LOOPS

3. `forEach()` Method for Arrays

Although not strictly an ES6 feature (introduced in ES5), the `forEach()` method is often used with arrow functions in ES6 for iterating over arrays:

```
const numbers = [1, 2, 3, 4, 5];  
numbers.forEach(num => console.log(num));
```

FUNCTIONS

1. Arrow Functions

Arrow functions provide a more concise syntax compared to traditional function expressions.

They are especially useful for shorter functions and preserve the lexical 'this' binding from the surrounding context.

Basic syntax:

```
// ES5  
var double = function(x) {  
    return x * 2;  
};
```

```
// ES6 arrow function  
const double = x => x * 2;
```

FUNCTIONS

With multiple parameters:

javascript

```
const add = (a, b) => a + b;
```

With block body (for multiple statements):

javascript

```
const greet = name => {  
  console.log(`Hello, ${name}!`);  
  console.log(`How are you?`);  
};
```

FUNCTIONS

- Arrow functions are anonymous functions (the functions without a name and not bound with an identifier).
- They don't return any value and can declare without the function keyword.
- Arrow functions cannot be used as the constructors.
- The context within the arrow functions is lexically or statically defined.
- They are also called as Lambda Functions in different languages

ARROW FUNCTION SYNTAX

```
const functionName = (arg1, arg2, ?..) => { //body of the function }
```

There are three parts to an Arrow Function or Lambda Function:

- Parameters: Any function may optionally have the parameters.
- Fat arrow notation/lambda notation: It is the notation for the arrow (=>).
- Statements: It represents the instruction set of the function.

FUNCTIONS

2. Default Parameters

ES6 allows you to specify default values for function parameters, which are used when the argument is not provided or is 'undefined'.

```
// ES6
function greet(name = 'Guest') {
  console.log(`Hello, ${name}!`);
}

greet(); // Hello, Guest!
greet('Alice'); // Hello, Alice!
```

FUNCTIONS

3. Rest Parameters

Rest parameters allow functions to accept an indefinite number of arguments as an array.

```
// ES6
function sum(...numbers) {
    return numbers.reduce((acc, num) => acc + num, 0);
}

console.log(sum(1, 2, 3)); // 6
console.log(sum(4, 5, 6, 7)); // 22
```

FUNCTIONS

4. Spread Syntax

The spread syntax (...) allows an iterable (like an array) to be expanded into individual elements. It can be used in function calls and array literals.

```
// Using spread in function calls  
function multiply(x, y, z) {  
    return x * y * z;  
}  
  
const args = [2, 3, 4];  
console.log(multiply(...args)); //
```

Spread parameters

```
<script>
  var array1 = [10, 20, 30, 40, 50];
  var array2 = [60, 70, 80, 90, 100];
  var array3 = [...array1, ...array2];
  console.log(array3);
</script>
```

Output:

```
[10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
```

FUNCTIONS

5. arguments Object

In arrow functions, the 'arguments' object is not available. However, regular functions in ES6 still have access to 'arguments'.

```
// ES6
function foo() {
  console.log(arguments); // [1, 2, 3]
}

foo(1, 2, 3);
```

FUNCTIONS

6. 'Function' Constructor

The 'Function' constructor can now use the arrow function syntax in ES6.

```
// ES6  
const add = new Function('a', 'b', 'return a + b');  
console.log(add(2, 3)); // 5
```

FUNCTIONS

7. Method Properties

ES6 allows you to define methods in object literals using shorthand syntax.

```
// ES6
const person = {
  name: 'Alice',
  greet() {
    console.log(`Hello, ${this.name}!`);
  }
};

person.greet(); // Hello, Alice!
```


EVENTS

- ES6, or ECMAScript 2015, introduced several new features to JavaScript, including improvements in handling events.
- JavaScript is meant to add interactivity to your pages.
- JavaScript does this using a mechanism using events.
- On the occurrence of an event, the application executes a set of related tasks.
- The block of code that achieves this purpose is called the eventhandler.

EVENTS

```
button.addEventListener('click', () => {  
    console.log('Button clicked');  
});
```

In this example, `() => { ... }` is an arrow function used as an event handler for the 'click' event of a button.

onclick Event Type

```
<html>
  <head>
    <script type = "text/javascript">
      function sayHello() {
        document.write ("Hello World")
      }
    </script>
  </head>

  <body>
    <p> Click the following button and see result</p>
    <input type = "button" onclick = "sayHello()" value = "Say Hello" />
  </body>
</html>
```

OUTPUT

Click the following button and see result

Say Hello

HTML 5 Standard Events

Attribute	Value	Description
onblur	script	Triggers when the window loses focus
onchange	script	Triggers when an element changes
onclick	script	Triggers on a mouse click
oncontextmenu	script	Triggers when a context menu is triggered
ondblclick	script	Triggers on a mouse double-click
ondrag	script	Triggers when an element is dragged
onfocus	script	Triggers when the window gets focus
onformchange	script	Triggers when a form changes
onforminput	script	Triggers when a form gets user input

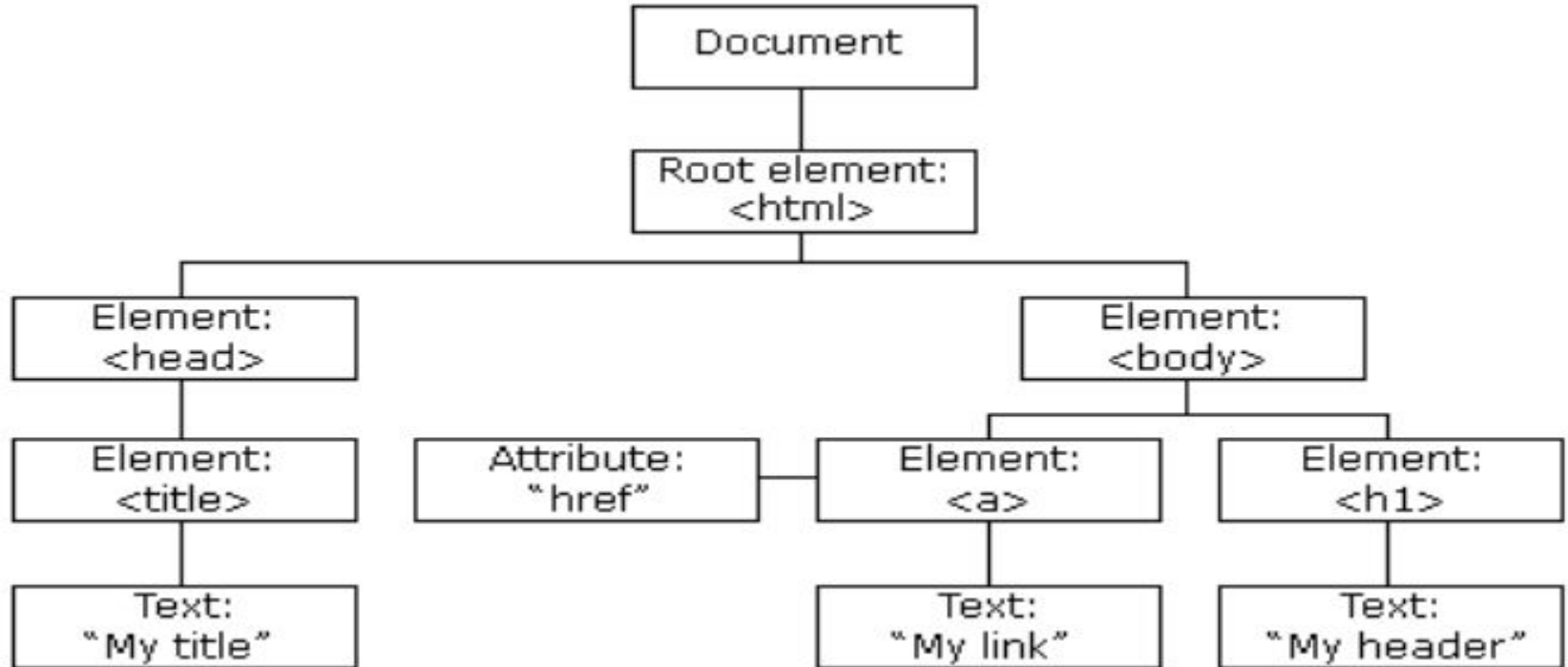
HTML 5 Standard Events

Attribute	Value	Description
oninput	script	Triggers when an element gets user input
oninvalid	script	Triggers when an element is invalid
onkeydown	script	Triggers when a key is pressed
onkeypress	script	Triggers when a key is pressed and released
onkeyup	script	Triggers when a key is released
onload	script	Triggers when the document loads
onmousemove	script	Triggers when the mouse pointer moves
onmouseout	script	Triggers when the mouse pointer moves out of an element
onmouseover	script	Triggers when the mouse pointer moves over an element
onsubmit	script	Triggers when a form is submitted

DOCUMENT OBJECT MODEL (DOM)

- The HTML DOM (Document Object Model)
- When a web page is loaded, the browser creates a Document Object Model of the page.
- The HTML DOM model is constructed as a tree of Objects:

DOCUMENT OBJECT MODEL (DOM) TREE



With DOM JAVASCRIPT can...

With the object model, JavaScript gets all the power it needs to create dynamic HTML:

- JavaScript can change all the HTML elements in the page
- JavaScript can change all the HTML attributes in the page
- JavaScript can change all the CSS styles in the page
- JavaScript can remove existing HTML elements and attributes
- JavaScript can add new HTML elements and attributes
- JavaScript can react to all existing HTML events in the page
- JavaScript can create new HTML events in the page

The document object

Finding HTML Elements

Method	Description
<code>document.getElementById(id)</code>	Find an element by element id
<code>document.getElementsByTagName(name)</code>	Find elements by tag name
<code>document.getElementsByClassName(name)</code>	Find elements by class name

The document object

Changing HTML Elements

Property

Description

`element.innerHTML = new html content` - Change the inner HTML of an element
`element.attribute = new value` - Change the attribute value of an HTML Element
`element.style.property = new style` - Change the style of an HTML element

Method

Description

`element.setAttribute(attribute, value)` - Change the attribute value of an HTML element

The document object

Adding and Deleting Elements

Property

Description

`document.createElement(element)`

Create an HTML element

`document.removeChild(element)`

Remove an HTML element

`document.appendChild(element)`

Add an HTML element

`document.replaceChild(new, old)`

Replace an HTML element

`document.write(text)`

Write into the HTML output stream

SETTING CSS STYLE USING JAVASCRIPT

The HTML DOM allows JavaScript to change the style of HTML elements.

- To change the style of an HTML element, use this syntax:

```
document.getElementById(id).style.property = new style
```

SETTING CSS STYLE USING JAVASCRIPT

```
<!DOCTYPE html>

<html>

<body>

<h2>JavaScript HTML DOM</h2>

<p>Changing the HTML style:</p>

<p id="p1">Hello World!</p>

<p id="p2">Hello World!</p>

<script>
```

```
document.getElementById("p2").style.color = "blue";

document.getElementById("p2").style.fontFamily =
"Arial";

document.getElementById("p2").style.fontSize = "larger";

</script>

</body>

</html>
```

OUTPUT:

JavaScript HTML DOM

Changing the HTML style:

Hello World!

Hello World!

CLASSES

ECMAScript 6 (ES6) introduced a new syntax for creating classes in JavaScript. This new class syntax is syntactic sugar over JavaScript's existing prototype-based inheritance model but makes it easier to work with and understand.

CLASSES

Defining a Class

```
class Person {  
  // Constructor method  
  constructor(name, age) {  
    this.name = name;  
    this.age = age;  
  }  
  
  // Method  
  greet() {  
    console.log(`Hello, my name is ${this.name} and I am ${this.age} years old.`);  
  }  
}
```

CLASSES

```
// Static method
static species() {
    return 'Homo sapiens';
}

// Creating an instance of the class
const person1 = new Person('Alice', 30);

// Accessing properties and methods
person1.greet(); // Output: Hello, my name is Alice and I am 30 years old.
console.log(Person.species()); // Output: Homo sapiens
```


INHERITANCE

You can also create subclasses that inherit from other classes using the `'extends'` keyword and call the parent class's constructor using the `'super'` keyword.

We can have single and multi-level inheritance in ECMAScript-6.

INHERITANCE

```
class Employee extends Person {  
    constructor(name, age, jobTitle) {  
        // Call the parent class's constructor  
        super(name, age);  
        this.jobTitle = jobTitle;  
    }  
  
    // Overriding a method  
    greet() {  
        console.log(`Hello, my name is ${this.name}, I am ${this.age} years old and I work`  
    }  
}
```



INHERITANCE

```
// Creating an instance of the subclass
const employee1 = new Employee('Bob', 25, 'Software Engineer');

// Accessing properties and methods
employee1.greet(); // Output: Hello, my name is Bob, I am 25 years old and I work as a So
```

Getters and Setters

ES6 classes also support getter and setter methods, which can be used to define computed properties.

```
class Rectangle {  
  constructor(width, height) {  
    this.width = width;  
    this.height = height;  
  }  
}
```

Getters and Setters

```
// Getter
get area() {
    return this.width * this.height;
}

// Setter
set area(value) {
    this.width = Math.sqrt(value);
    this.height = Math.sqrt(value);
}
}
```

Getters and Setters

```
const rect = new Rectangle(4, 5);  
  
console.log(rect.area); // Output: 20  
  
rect.area = 25;  
  
console.log(rect.width); // Output: 5  
console.log(rect.height); // Output: 5
```

Getters and Setters

```
// Create an object:
```

```
const person = {  
  firstName: "John",  
  lastName: "Doe",  
  language: "en",  
  get lang() {  
    return this.language;  
  }  
};
```

```
// Display data from the object using a getter:
```

```
document.getElementById("demo").innerHTML = person.lang;
```

Getters and Setters

```
const person = {  
  firstName: "John",  
  lastName: "Doe",  
  language: "",  
  set lang(lang) {  
    this.language = lang;  
  }  
};  
  
// Set an object property using a setter:  
person.lang = "en";  
  
// Display data from the object:  
document.getElementById("demo").innerHTML = person.language;
```


JavaScript Function or Getter?

Example 1

```
const person = {  
  firstName: "John",  
  lastName: "Doe",  
  fullName: function() {  
    return this.firstName + " " + this.lastName;  
  }  
};  
  
// Display data from the object using a method:  
document.getElementById("demo").innerHTML = person.fullName();
```

JavaScript Function or Getter?

Example 2

```
const person = {  
  firstName: "John",  
  lastName: "Doe",  
  get fullName() {  
    return this.firstName + " " + this.lastName;  
  }  
};  
  
// Display data from the object using a getter:  
document.getElementById("demo").innerHTML = person.fullName;
```

INHERITANCE

Private Fields

ES6 classes also support private fields (available in ES2022 and later), which can be defined using the # symbol. Private fields are only accessible within the class body.

```
class Car {  
  #brand;  
  
  constructor(brand, model) {  
    this.#brand = brand;  
    this.model = model;  
  }  
}
```

INHERITANCE

```
    getBrand() {  
        return this.#brand;  
    }  
}
```

```
const car1 = new Car('Toyota', 'Corolla');  
console.log(car1.getBrand()); // Output: Toyota
```

```
// Accessing the private field directly will result in an error  
// console.log(car1.#brand); // SyntaxError: Private field '#brand' must be declared
```

INHERITANCE

Example in HTML

Here is an example of how you might use an ES6 class in an HTML file:

```
<!DOCTYPE html>
<html>
<head>
  <title>ES6 Class Example</title>
</head>
<body>
```

```
<script>
  class Person {
    constructor(name, age) {
      this.name = name;
      this.age = age;
    }

    greet() {
      console.log(`Hello, my name is ${this.name} and I am ${this.age} years old.`);
    }
  }

  const person1 = new Person('Alice', 30);
  person1.greet(); // Output: Hello, my name is Alice and I am 30 years old.
</script>

</body>
</html>
```



ITERATORS AND GENERATORS

- Iterators and generators are advanced concepts in JavaScript introduced with ECMAScript 6 (ES6).
- They are powerful tools for working with data structures and controlling iteration behavior.

ITERATORS

In JavaScript, iterators provide a standard way to traverse collections of data.

They offer a way to work with data structures like arrays, maps, sets, and custom collections in a consistent manner.

ITERATORS - Concepts

Iterator:

- An object that provides a way to access elements in a collection one at a time.
- Must implement a `next()` method that returns an object with two properties: `value` (the current element) and `done` (a boolean indicating if there are more elements).

ITERATORS

Iterable:

- An object that has a method `Symbol.iterator` which returns an iterator.
- Examples include arrays, strings, maps, and sets.
- The `[symbol.iterator]()` can be used to retrieve and iterator object

ITERATORS

Built-in Iterables

Arrays: Arrays are iterable by default.

```
const array = [1, 2, 3, 4];  
const iterator = array[Symbol.iterator]();  
  
console.log(iterator.next()); // { value: 1, done: false }  
console.log(iterator.next()); // { value: 2, done: false }  
console.log(iterator.next()); // { value: 3, done: false }  
console.log(iterator.next()); // { value: 4, done: false }  
console.log(iterator.next()); // { value: undefined, done: true }
```

ITERATORS

Strings: Strings are also iterable.

```
const str = "hello";
const iterator = str[Symbol.iterator]();

console.log(iterator.next()); // { value: 'h', done: false }
console.log(iterator.next()); // { value: 'e', done: false }
console.log(iterator.next()); // { value: 'l', done: false }
console.log(iterator.next()); // { value: 'l', done: false }
console.log(iterator.next()); // { value: 'o', done: false }
console.log(iterator.next()); // { value: undefined, done: true }
```

ITERATORS

Maps: Maps have their own iterators for keys, values, and entries.

```
const map = new Map();
map.set('a', 1);
map.set('b', 2);

const iterator = map[Symbol.iterator]();

console.log(iterator.next()); // { value: ['a', 1], done: false }
console.log(iterator.next()); // { value: ['b', 2], done: false }
console.log(iterator.next()); // { value: undefined, done: true }
```

ITERATORS

Sets: Sets are also iterable.

```
const set = new Set([1, 2, 3]);  
const iterator = set[Symbol.iterator]();  
  
console.log(iterator.next()); // { value: 1, done: false }  
console.log(iterator.next()); // { value: 2, done: false }  
console.log(iterator.next()); // { value: 3, done: false }  
console.log(iterator.next()); // { value: undefined, done: true }
```

ITERATORS

Custom Iterables (eg: Class)

You can define your own iterable objects by implementing the `Symbol.iterator` method.

```
class MyIterable {  
  constructor(items) {  
    this.items = items;  
  }  
}
```

```
[Symbol.iterator]() {  
  let index = 0;  
  const items = this.items;  
  return {  
    next() {  
      if (index < items.length) {  
        return { value: items[index++], done: false };  
      } else {  
        return { value: undefined, done: true };  
      }  
    }  
  };  
}
```


ITERATORS

```
const myIterable = new MyIterable([10, 20, 30]);  
for (let value of myIterable) {  
    console.log(value);  
}  
  
// Output:  
// 10  
// 20  
// 30
```

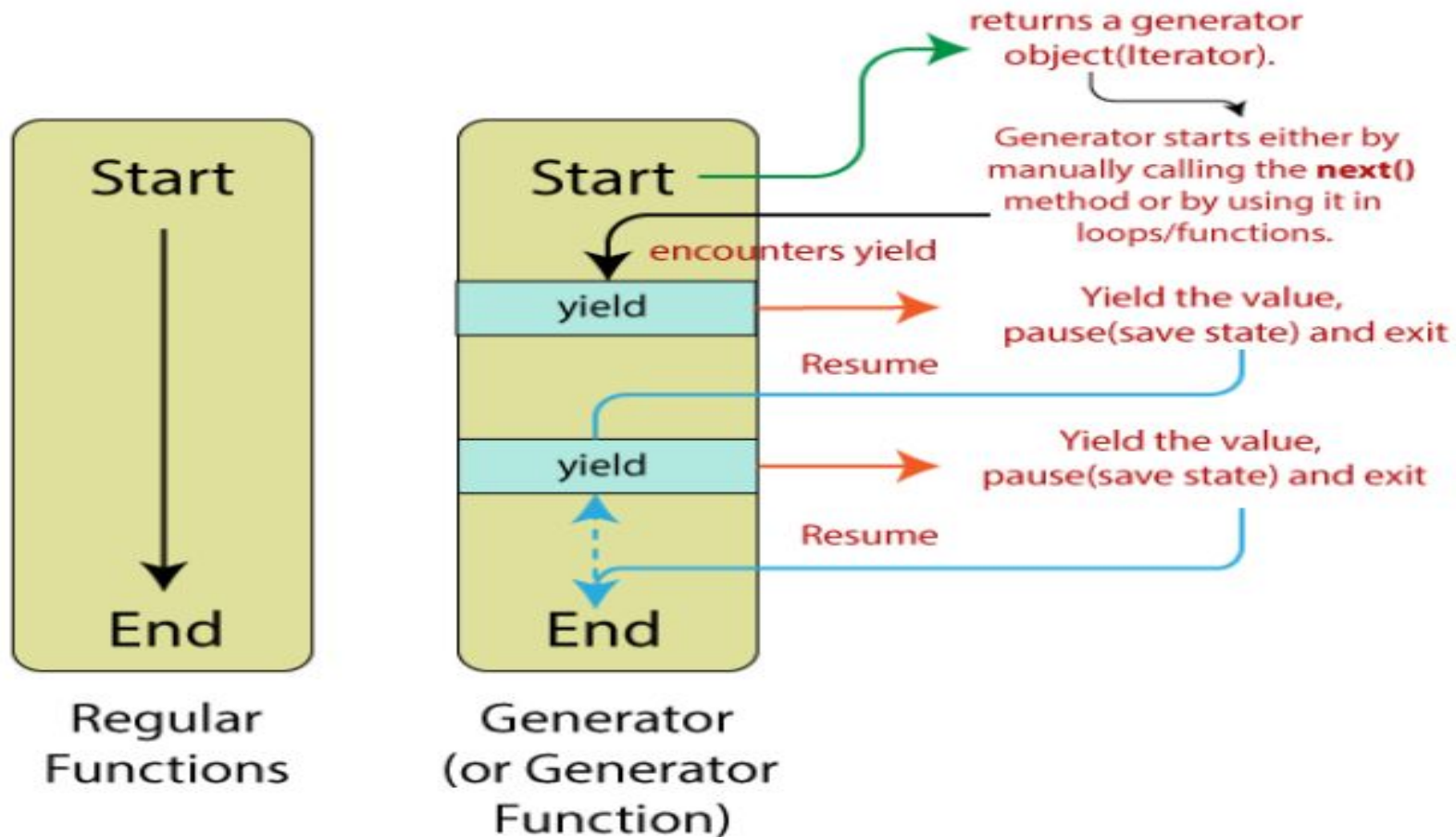
ITERATORS

Why use iterator instead of for loop JavaScript?

for loops are suitable for simple iteration, especially when direct access to indices is beneficial. Iterators, on the other hand, are powerful when dealing with safe removal, forward-only traversal, and when working with various collection types.

GENERATORS

- ES6 generator is a different kind of function that may be paused in the middle either one or many times and can be resumed later.
- When the standard function is called, the control rests with the called function until it returns, but the generator in ES6 allows the caller function to control the execution of a called function.



GENERATORS

A generator is somewhere identical to the regular function except that:

- When the generator gets called, it doesn't run its code. Instead, it returns a special object which is called as 'Generator Object' for managing the execution.
- The generator function can return (or yield) the control back to the caller at any point.
- Unlike the regular function, the generator can return (or yield) the multiple values, one after another, on the requirement.

Basic Generator Example

Generators provide a more concise way to create iterators using the `function*` syntax.

```
function* simpleGenerator() {  
  yield 1;  
  yield 2;  
  yield 3;  
}  
  
const gen = simpleGenerator();  
console.log(gen.next().value); // Output: 1  
console.log(gen.next().value); // Output: 2  
console.log(gen.next().value); // Output: 3  
console.log(gen.next().done);  // Output: true
```

Using Generators with 'for...of'

```
function* numberGenerator() {  
  yield 1;  
  yield 2;  
  yield 3;  
}  
  
for (const num of numberGenerator()) {  
  console.log(num); // Output: 1, 2, 3  
}
```

Generator Example with Arguments and Return

```
function* rangeGenerator(start, end) {  
  for (let i = start; i <= end; i++) {  
    yield i;  
  }  
  return end + 1;  
}
```

```
const range = rangeGenerator(1, 3);
```

```
console.log(range.next()); // Output: { value: 1, done: false }  
console.log(range.next()); // Output: { value: 2, done: false }  
console.log(range.next()); // Output: { value: 3, done: false }  
console.log(range.next()); // Output: { value: 4, done: true }
```


Choosing Between Iterators and Generators

Use Iterators when you need a custom iteration protocol for your data structures and are comfortable managing the iteration state manually.

Use Generators for simpler, more readable iteration logic, especially when you need to pause and resume execution or handle complex sequences of values.

Callbacks

Asynchronous actions are actions that we initiate now and finish later. Eg `setTimeout`. Synchronous action are actions that initiate and finish one after the other.

Callback functions

A callback function is a function passed into another function as an argument, which is then invoked inside the outer function to complete an action.

PROMISE

- Promises are a key feature of ES6 (ECMAScript 2015) that allow for better handling of asynchronous operations.
- They provide a cleaner, more robust way to work with asynchronous code compared to callbacks.

PROMISE

A promise is basically created when we are unsure of whether or not the assigned task will be completed.

The promise object represent the eventual completion (or failure) of an async (asynchronous) operation and its resulting value.

PROMISE

A promise is always in one of the following states:

- Fulfilled: Action related to the promise succeeded
- Rejected : Action related to the promise failed
- Pending : Promise is still pending ie., not fulfilled or rejected yet.
- Settled : Promise has fulfilled or rejected.

Syntax:

```
const promise = new Promise(resolve, reject)=> {...};
```

PROMISE

Example:

```
const myPromise = new Promise((resolve, reject) => {  
  If (math.random()>0) {  
    resolve('hello, I am Positive number !');  
  }  
  reject(new Error('I failed some times'));  
})
```

Creating a Promise

- A promise is created using the 'Promise' constructor, which takes a function (the executor) with two parameters: 'resolve' and 'reject'.
- These parameters are functions that you call to settle the promise.

```
const myPromise = new Promise((resolve, reject) => {  
  // Asynchronous operation  
  let success = true; // Change this to false to test rejection  
  
  if (success) {  
    resolve("Operation was successful!");  
  } else {  
    reject("Operation failed.");  
  }  
});
```

Handling Promises

You handle a promise using its `'then'`, `'catch'`, and `'finally'` methods:

- `'then'` is used for handling a successful outcome.
- `'catch'` is used for handling errors.
- `'finally'` is used for code that should run regardless of the outcome.

Handling Promises

```
myPromise
  .then(result => {
    console.log(result); // Output: Operation was successful!
  })
  .catch(error => {
    console.error(error); // Output: Operation failed.
  })
  .finally(() => {
    console.log("This will run regardless of the promise outcome.");
  });
```

Chaining Promises

- Promises can be chained to handle a sequence of asynchronous operations.
- Each 'then' method returns a new promise, allowing you to chain additional 'then' methods.

```
const promiseChain = new Promise((resolve, reject) => {  
  resolve(10);  
});
```

```
promiseChain  
  .then(result => {  
    console.log(result); // Output: 10  
    return result * 2;  
  })  
  .then(result => {  
    console.log(result); // Output: 20  
    return result * 3;  
  })  
  .then(result => {  
    console.log(result); // Output: 60  
  })  
  .catch(error => {  
    console.error(error);  
  });
```



Promises with Asynchronous Functions

Promises are often used with asynchronous functions like 'fetch':

```
fetch('https://api.example.com/data')
  .then(response => response.json())
  .then(data => {
    console.log(data);
  })
  .catch(error => {
    console.error('Error:', error);
  });
```

Promise Methods

Promise.all

`Promise.all` takes an array of promises and returns a single promise that resolves when all the promises have resolved, or rejects if any promise rejects.

```
const promise1 = Promise.resolve(3);
const promise2 = 42;
const promise3 = new Promise((resolve, reject) => {
  setTimeout(resolve, 100, 'foo');
});

Promise.all([promise1, promise2, promise3]).then(values => {
  console.log(values); // Output: [3, 42, "foo"]
});
```

Promise Methods

Promise.race

`Promise.race` takes an array of promises and returns a single promise that resolves or rejects as soon as one of the promises resolves or rejects.

```
const promise1 = new Promise((resolve, reject) => {
  setTimeout(resolve, 500, 'one');
});

const promise2 = new Promise((resolve, reject) => {
  setTimeout(resolve, 100, 'two');
});

Promise.race([promise1, promise2]).then(value => {
  console.log(value); // Output: "two"
});
```

Promise Methods

Promise.allSettled

`Promise.allSettled` takes an array of promises and returns a promise that resolves after all of the given promises have either resolved or rejected, with an array of objects that each describe the outcome of each promise.

```
const promise1 = Promise.resolve(3);
const promise2 = new Promise((resolve, reject) => {
  setTimeout(reject, 100, 'error');
});
const promise3 = Promise.resolve(42);

Promise.allSettled([promise1, promise2, promise3]).then(results => {
  results.forEach((result) => console.log(result.status));
  // Output:
  // "fulfilled"
  // "rejected"
  // "fulfilled"
});
```


Promise Methods

Promise.any

`Promise.any` takes an array of promises and returns a single promise that resolves as soon as one of the promises in the iterable resolves. If no promises in the iterable resolve, it returns a promise that is rejected with an `'AggregateError'`.

Promise Methods

```
const promise1 = Promise.reject('error1');
const promise2 = Promise.reject('error2');
const promise3 = Promise.resolve(42);

Promise.any([promise1, promise2, promise3]).then(value => {
  console.log(value); // Output: 42
}).catch(error => {
  console.error(error);
});
```

Example in HTML

```
<!DOCTYPE html>
<html>
<head>
  <title>Promises Example</title>
</head>
<body>

<script>
  function asyncOperation() {
    return new Promise((resolve, reject) => {
      setTimeout(() => {
        let success = true; // Change to false to test rejection
```

```
        if (success) {
            resolve("Operation completed successfully!");
        } else {
            reject("Operation failed.");
        }
    }, 1000);
});
}
```

```
asyncOperation()
    .then(result => {
        console.log(result); // Output after 1 second: Operation completed successful
    })
    .catch(error => {
        console.error(error); // Output if rejected: Operation failed.
    });
```

</script>

</body>

</html>



Promise Methods

- Promises provide a powerful way to handle asynchronous operations, making your code cleaner and more manageable.
- They allow you to chain operations, handle errors gracefully, and work with multiple asynchronous tasks simultaneously.

CLIENT-SERVER COMMUNICATION

- Client-server communication is the exchange of data between a client (usually a web browser) and a server over a network.
- This is fundamental to web applications, enabling features like fetching data from a server, submitting form data, and real-time updates.

FETCH

- The 'fetch' API is a modern and versatile way to make network requests in JavaScript.
- It provides a more powerful and flexible feature set than the older 'XMLHttpRequest' and returns promises, making it easier to handle asynchronous operations.

Basic Usage

The basic syntax of 'fetch' involves calling 'fetch(url, options)' and handling the promise it returns.

URL: it is the URL to which the request is to be made.

Options: it is an array of properties. It is an optional parameter.

Return value: it returns a promises whether it is resolved or not. The return data can be of the format JSON or XML.


```
fetch('https://api.example.com/data')
  .then(response => {
    if (!response.ok) {
      throw new Error('Network response was not ok');
    }
    return response.json();
  })
  .then(data => {
    console.log(data);
  })
  .catch(error => {
    console.error('There was a problem with the fetch operation:', error);
  });
```

EXPLANATION

In this example:

1. `fetch` is called with the URL of the resource.
2. The response object is checked to see if the request was successful.
3. The `json` method of the response object is called to parse the JSON data.
4. The parsed data is logged to the console.
5. Errors are caught and logged.

THANK YOU

MU QS.

Differentiate between es5 and es6.

Explain promises with an example.

Explain arrow functions with suitable example

What is Asynchronous Programming?